

Кременчуцький національний університет імені Михайла Остроградського
(повне найменування вищого навчального закладу)

Кафедра комп'ютерної інженерії та електроніки
(повна назва кафедри)

ЛАБОРАТОРНА РОБОТА № 6

Тема: Асоціативні структури даних: словники та множини.

Мета роботи: Опанувати словники, множини та частотний аналіз у Python.

Студента III курсу KI-24-1 групи

Ступінь вищої освіти бакалавр
(бакалавр, магістр)

Спеціальність 123 – «Комп'ютерна інженерія»

Освітня програма Програмування на Python. Базовий рівень

Огоновського Олександра Євгенійовича

м. Кременчук 2026 рік

1.1 Порядок виконання роботи

1.1.1 Постановка індивідуального завдання

Варіант 15 присвячений інвентаризації поштових релеїв і фільтрів spam. Реєстр має вкладену структуру Relay_IP -> {Domain, SPF_Records_Set, DKIM_Status}. У роботі виконано CRUD-операції словника, безпечний доступ через get() і setdefault(), побудовано інвертований індекс SPF-механізмів за допомогою Dict Comprehension, а також проведено множинний аудит адрес відправників.

Фактична множина IP-адрес відправників A порівнюється з нормативним профілем MX/SPF B . Обчислюються дозволені адреси $A \cap B$, недозволені $A \setminus B$, очікувані адреси $B \setminus A$ та коефіцієнт Жаккара $J(A, B) = |A \cap B| / |A \cup B|$. Для відхилення подій Counter визначає найчастіші джерела spam-повідомлень.

Таблиця 1.1 – Структура запису поштового релея

Поле	Тип значення	Призначення
Relay_IP	str	Ключ словника; IPv4-адреса поштового релея.
domain	str	Домен, який обслуговує релей.
spf_records	set[str]	Унікальні SPF-механізми; доповнюються через setdefault().
dkim_status	str	Стан підпису DKIM: PASS, FAIL або UNKNOWN.
Counter spam	Counter[str]	Частоти відхилення IP-адрес відправників.

1.1.2 Використані алгоритми

Функція add_or_update_relay створює новий запис або оновлює наявний через dict.update(). Тимчасове реле видаляється безпечно методом pop(key, None). Множина SPF-механізмів гарантує унікальність значень; Dict Comprehension формує відображення SPF-механізм -> список доменів. Середня складність доступу до ключа у словнику та перевірки належності множині становить $O(1)$.

У модулі аудиту операції intersection(), difference() та union() не змінюють початкові множини. Результат їх застосування використовується для розрахунку індексу Жаккара. Counter агрегує IP-адреси відхилення відправників і метод most_common(3) повертає три найчастіші джерела подій.

1.2 Результати виконання

1.2.1 Реєстр релеїв і SPF-індекс

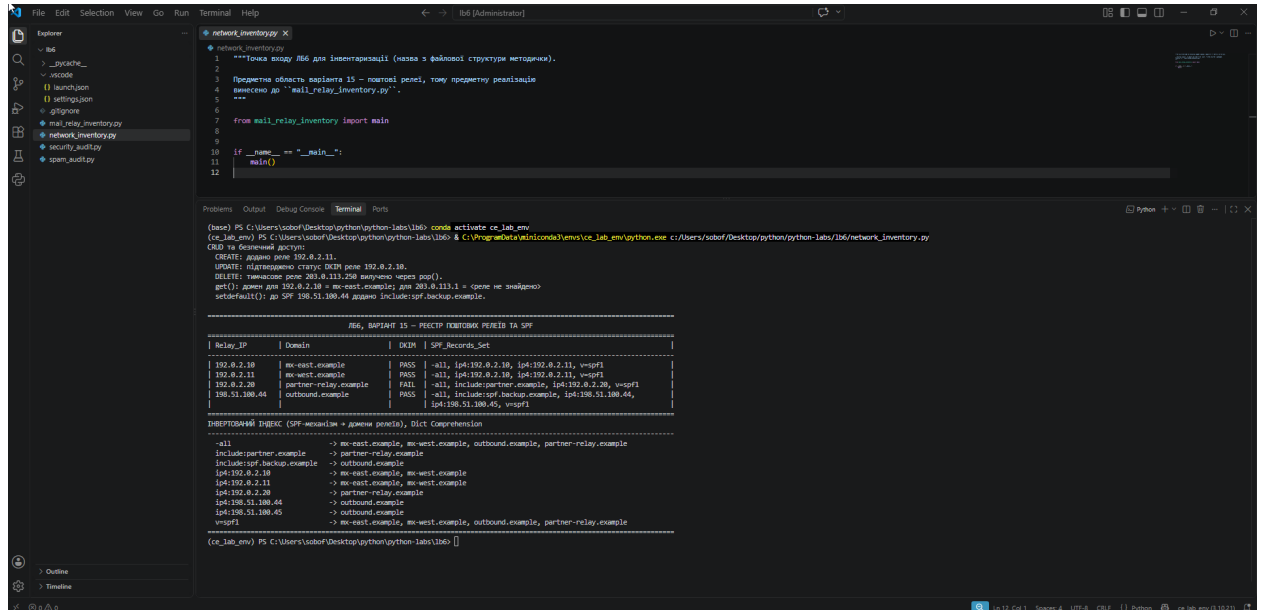


Рисунок 1.1 – Скриншот CRUD-операцій, реєстру релеїв і SPF-індексу

У консолі показано створення, оновлення та видалення запису, безпечний пошук домену й доповнення SPF-набору. Після цього виведено чотири релеї та інвертований індекс, який пов'язує кожен SPF-механізм зі списком доменів.

1.2.2 Аудит MX/SPF і Counter

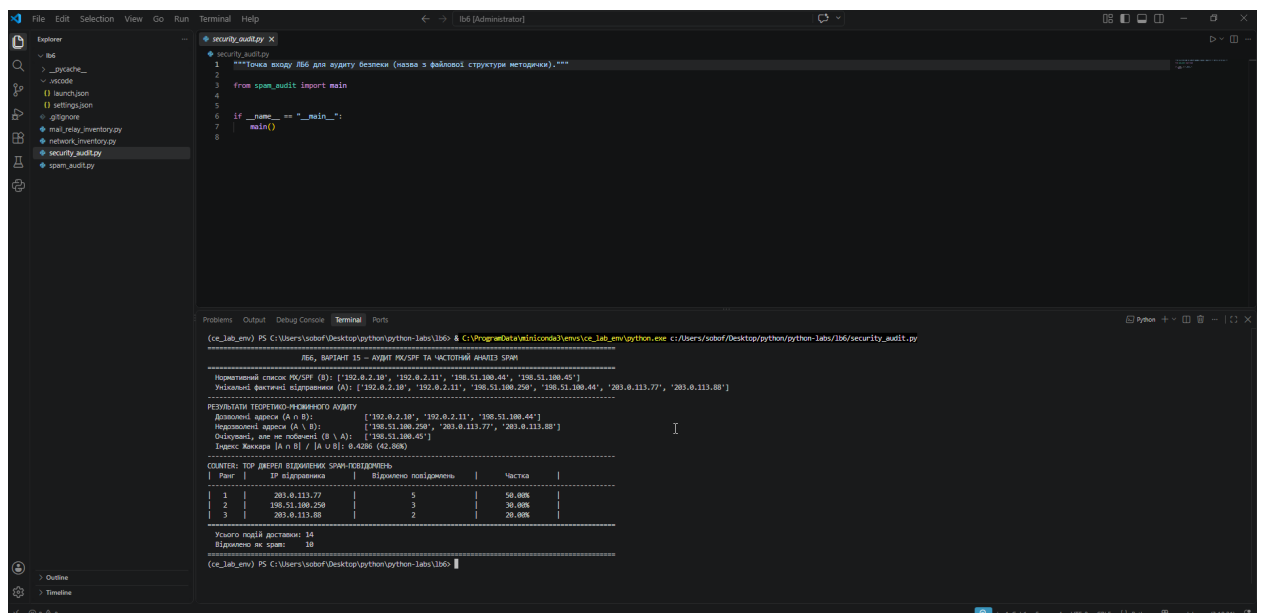


Рисунок 1.2 – Скриншот множинного аудиту MX/SPF і частотного аналізу спам

Аудит виявив три дозволені адреси, три недозволені та одну очікувану, але не побачену адресу. Індекс Жаккара дорівнює 0.4286 (42.86%). Counter ранжує 203.0.113.77 як найчастіше джерело: п'ять із десяти відхилених повідомлень.

1.3 Програмний код

1.3.1 Точка входу інвентаризації

Файл network_inventory.py

```
"""Точка входу ЛБ6 для інвентаризації (назва з файлової структури методички).

Предметна область варіанта 15 – поштові релеї, тому предметну реалізацію
винесено до ``mail_relay_inventory.py``.
"""

from mail_relay_inventory import main

if __name__ == "__main__":
    main()
```

1.3.2 Модуль реєстру поштових релеїв

Файл mail_relay_inventory.py

```
"""Laboratory work 6, variant 15: mail-relay inventory using dictionaries.

The nested registry implements the required mapping
``Relay_IP -> {Domain, SPF_Records_Set, DKIM_Status}``. Its data is kept
deterministic so the console report is suitable for a repeatable lab run.
"""

from __future__ import annotations
from textwrap import wrap

def create_initial_relay_database() -> dict[str, dict[str, object]]:
    """Return the initial nested dictionary of mail relays and SPF policies."""
    return {
        "192.0.2.10": {
            "domain": "mx-east.example",
            "spf_records": {"v=spf1", "ip4:192.0.2.10", "ip4:192.0.2.11", "-all"},
            "dkim_status": "PASS",
        },
        "192.0.2.20": {
            "domain": "partner-relay.example",
            "spf_records": {"v=spf1", "ip4:192.0.2.20", "include:partner.example", "-all"},
            "dkim_status": "FAIL",
        },
        "198.51.100.44": {
            "domain": "outbound.example",
            "spf_records": {
                "v=spf1",
                "ip4:198.51.100.44",
                "ip4:198.51.100.45",
                "-all",
            },
            "dkim_status": "PASS",
        },
    }

def add_or_update_relay(
    relay_database: dict[str, dict[str, object]],
    relay_ip: str,
    relay_info: dict[str, object],
) -> str:
    """Create a relay or update its fields through :meth:`dict.update`."""

    if relay_ip in relay_database:
        relay_database[relay_ip].update(relay_info)
        return "UPDATE"

    relay_database[relay_ip] = relay_info
    return "CREATE"
```

```

def decommission_relay(
    relay_database: dict[str, dict[str, object]], relay_ip: str
) -> dict[str, object] | None:
    """Safely remove a temporary or retired relay with :meth:`dict.pop`."""

    return relay_database.pop(relay_ip, None)

def get_relay_domain(relay_database: dict[str, dict[str, object]], relay_ip: str) -> str:
    """Look up a domain safely with :meth:`dict.get` without risking ``KeyError``."""

    relay = relay_database.get(relay_ip)
    if relay is None:
        return "<реле не знайдено>"
    return str(relay.get("domain", "<домен не задано>"))

def add_spf_mechanism(
    relay_database: dict[str, dict[str, object]], relay_ip: str, mechanism: str
) -> None:
    """Add an SPF mechanism using ``setdefault`` for the nested set field."""

    relay = relay_database.get(relay_ip)
    if relay is None:
        raise KeyError(f"Неможливо доповнити SPF: реле {relay_ip} не існує.")

    spf_records = relay.setdefault("spf_records", set())
    if not isinstance(spf_records, set):
        raise TypeError("Поле spf_records має бути множиною.")
    spf_records.add(mechanism)

def build_spf_inverted_index(
    relay_database: dict[str, dict[str, object]],
) -> dict[str, list[str]]:
    """Build ``SPF mechanism -> relay domains`` with a Dict Comprehension."""

    all_mechanisms = {
        mechanism
        for relay in relay_database.values()
        for mechanism in relay.get("spf_records", set())
        if isinstance(mechanism, str)
    }

    return {
        mechanism: sorted(
            str(relay.get("domain", "<домен не задано>"))
            for relay in relay_database.values()
            if mechanism in relay.get("spf_records", set())
        )
        for mechanism in sorted(all_mechanisms)
    }

def format_spf_records(relay: dict[str, object]) -> str:
    """Format the stored SPF set in a stable order for a console report."""

    records = relay.get("spf_records", set())
    if not isinstance(records, set):
        return "<помилковий тип SPF>"
    return ", ".join(sorted(str(record) for record in records))

def print_inventory_report(
    relay_database: dict[str, dict[str, object]], inverted_index: dict[str, list[str]]
) -> None:
    """Print the nested-dictionary inventory and its SPF inverted index."""

    separator = "=" * 119
    row_separator = "-" * 119
    print(separator)
    print(f"{'/ЛБ6, БАПІАHT 15 – РЕЄСТР ПОШТОВИХ РЕЛЕЇВ ТА SPF':^119}")
    print(separator)
    print(f"| {'Relay_IP':<15} | {'Domain':<25} | {'DKIM':^6} | {'SPF_Records_Set':<60} |")
    print(row_separator)

    for relay_ip, relay in sorted(relay_database.items()):
        spf_lines = wrap(
            format_spf_records(relay), width=60, break_long_words=False, break_on_hyphens=False
        ) or [""]
        print(f"| {relay_ip:<15} | {str(relay.get('domain', '')):<25} | "
              f"{str(relay.get('dkim_status', 'UNKNOWN')):^6} | {spf_lines[0]:<60} |")
        for spf_line in spf_lines[1:]:
            print(f"| {'':15} | {'':25} | {'':6} | {spf_line:<60} |")

    print(separator)

```

```

print("ІНВЕРТОВАНИЙ ІНДЕКС (SPF-механізм → домени релеїв), Dict Comprehension")
print(row_separator)
for mechanism, domains in inverted_index.items():
    print(f" {mechanism:<28} -> {' ', '.join(domains)}")
print(separator)

def main() -> None:
    """Demonstrate CRUD, safe access and a Dict Comprehension on the registry."""

    relay_database = create_initial_relay_database()

    create_action = add_or_update_relay(
        relay_database,
        "192.0.2.11",
        {
            "domain": "mx-west.example",
            "spf_records": {"v=spf1", "ip4:192.0.2.10", "ip4:192.0.2.11", "-all"},
            "dkim_status": "PASS",
        },
    )
    update_action = add_or_update_relay(
        relay_database, "192.0.2.10", {"dkim_status": "PASS"}
    )
    add_or_update_relay(
        relay_database,
        "203.0.113.250",
        {
            "domain": "retired-test-relay.example",
            "spf_records": {"v=spf1", "ip4:203.0.113.250", "-all"},
            "dkim_status": "UNKNOWN",
        },
    )
    removed_relay = decommission_relay(relay_database, "203.0.113.250")
    add_spf_mechanism(relay_database, "198.51.100.44", "include:spf.backup.example")

    print("CRUD та безпечний доступ:")
    print(f" {create_action}: додано реле 192.0.2.11.")
    print(f" {update_action}: підтверджено статус DKIM реле 192.0.2.10.")
    print(
        " DELETE: "
        + ("тимчасове реле 203.0.113.250 вилучено через pop()." if removed_relay else "не
виконано.")
    )
    print(
        " get(): домен для 192.0.2.10 = "
        f"{get_relay_domain(relay_database, '192.0.2.10')}; "
        f"для 203.0.113.1 = {get_relay_domain(relay_database, '203.0.113.1')}"
    )
    print(" setdefault(): до SPF 198.51.100.44 додано include:spf.backup.example.")
    print()

    inverted_index = build_spf_inverted_index(relay_database)
    print_inventory_report(relay_database, inverted_index)

if __name__ == "__main__":
    main()

```

1.3.3 Точка входу аудиту безпеки

Файл security_audit.py

```

"""Точка входу ЛБ6 для аудиту безпеки (назва з файлової структури методички)."""

from spam_audit import main

if __name__ == "__main__":
    main()

```

1.3.4 Модуль аудиту spam-подій

Файл spam_audit.py

```

"""Laboratory work 6, variant 15: SPF/MX sender audit and spam frequency.

The module compares observed sender addresses with a deterministic allow-list
derived from MX/SPF policy, then ranks rejected senders using
``collections.Counter``.
"""

from __future__ import annotations

```

```

from collections import Counter

# Sender addresses allowed by the configured MX/SPF policy.
AUTHORIZED_MX_SPF_IPS: frozenset[str] = frozenset(
    {"192.0.2.10", "192.0.2.11", "198.51.100.44", "198.51.100.45"}
)

def audit_sender_addresses(
    observed_ips: set[str], authorized_ips: frozenset[str]
) -> dict[str, set[str] | float]:
    """Compare actual senders ``A`` with MX/SPF policy ``B`` using sets."""

    compliant = observed_ips.intersection(authorized_ips)
    unauthorized = observed_ips.difference(authorized_ips)
    missing_expected = authorized_ips.difference(observed_ips)
    union = observed_ips.union(authorized_ips)
    jaccard_similarity = len(compliant) / len(union) if union else 1.0

    return {
        "compliant": compliant,
        "unauthorized": unauthorized,
        "missing_expected": missing_expected,
        "jaccard_similarity": jaccard_similarity,
    }

def count_spam_attacks(blocked_sender_events: list[str]) -> Counter[str]:
    """Return a frequency map of rejected sender IPs via ``Counter``."""

    return Counter(blocked_sender_events)

def print_audit_report(
    received_sender_events: list[str], audit: dict[str, set[str] | float]
) -> None:
    """Print the set audit and Counter ranking in a deterministic form."""

    observed_ips = set(received_sender_events)
    blocked_events = [ip for ip in received_sender_events if ip not in AUTHORIZED_MX_SPF_IPS]
    frequency = count_spam_attacks(blocked_events)
    jaccard_similarity = audit["jaccard_similarity"]

    if not isinstance(jaccard_similarity, float):
        raise TypeError("Індекс Жаккара має бути дійсним числом.")

    separator = "=" * 104
    row_separator = "-" * 104
    print(separator)
    print(f"{'ЛБ6', ВАРИАНТ 15 – АУДИТ MX/SPF ТА ЧАСТОТНИЙ АНАЛІЗ SPAM':^104}")
    print(separator)
    print(f" Нормативний список MX/SPF (B): {sorted(AUTHORIZED_MX_SPF_IPS)}")
    print(f" Унікальні фактичні відправники (A): {sorted(observed_ips)}")
    print(row_separator)
    print("РЕЗУЛЬТАТИ ТЕОРЕТИКО-МНОЖИННОГО АУДИТУ")
    print(f" Дозволені адреси (A ∩ B): {sorted(audit['compliant'])}")
    print(f" Недозволені адреси (A \\ B): {sorted(audit['unauthorized'])}")
    print(f" Очікувані, але не побачені (B \\ A): {sorted(audit['missing_expected'])}")
    print(
        f" Індекс Жаккара |A ∩ B| / |A ∪ B|: "
        f"{jaccard_similarity:.4f} ({jaccard_similarity * 100:.2f}%) "
    )
    print(row_separator)
    print("COUNTER: ТОП ДЖЕРЕЛ ВІДХИЛЕНИХ SPAM-ПОВІДОМЛЕНЬ")
    print(f"| {'Ранг':^6} | {'IP відправника':^25} | {'Відхилено повідомлень':^28} | {'Частка':^18} |")

    print(row_separator)

    total_blocked = len(blocked_events)
    for rank, (sender_ip, count) in enumerate(frequency.most_common(3), start=1):
        share = (count / total_blocked) * 100 if total_blocked else 0.0
        share_text = f"{share:.2f}%"
        print(f"| {rank:^6d} | {sender_ip:^25} | {count:^28d} | {share_text:^18} |")

    print(separator)
    print(f" Усього подій доставки: {len(received_sender_events)}")
    print(f" Відхилено як spam: {total_blocked}")
    print(separator)

def main() -> None:
    """Run the sender-policy comparison and rank all rejected senders."""

    received_sender_events = [
        "192.0.2.10",
        "192.0.2.10",
    ]

```

```

        "192.0.2.11",
        "198.51.100.44",
        "203.0.113.77",
        "203.0.113.77",
        "203.0.113.77",
        "203.0.113.77",
        "203.0.113.77",
        "198.51.100.250",
        "198.51.100.250",
        "198.51.100.250",
        "203.0.113.88",
        "203.0.113.88",
    ]
    audit = audit_sender_addresses(set(received_sender_events), AUTHORIZED_MX_SPF_IPS)
    print_audit_report(received_sender_events, audit)

if __name__ == "__main__":
    main()

```

1.4 Відповіді на контрольні питання

1.4.1 Компактний словник CPython

Compact Dict зберігає розріджену таблицю `indices` окремо від щільного масиву `entries`. У `indices` розташовані індекси записів або позначки порожніх комірок, а `entries` містить пари ключ-значення у порядку додавання. Під час зміни розміру переобчислюється таблиця `indices`, проте порядок щільного масиву зберігається.

1.4.2 Контракт хешованості

Хешований об'єкт має мати незмінний хеш протягом свого життя та виконувати правило: якщо $a == b$, то $\text{hash}(a) == \text{hash}(b)$. `list` і `set` змінювані, тому не мають стабільного хешу. `tuple` може бути ключем, якщо всі його елементи хешовані; `frozenset` незмінний і також може бути ключем або елементом множини.

1.4.3 `dict.get()` і `dict.setdefault()`

`get(key, default)` повертає значення або `default`, не змінюючи словник. `setdefault(key, default)` також повертає значення, але вставляє `default`, коли ключ відсутній. `setdefault` доцільний для накопичення списків або множин у вкладених структурах, зокрема для додавання SPF-механізмів.

1.4.4 Коефіцієнт Жаккара

$J(A, B) = |A \cap B| / |A \cup B|$ показує частку спільних елементів серед усіх унікальних елементів двох множин. Значення 1 означає повну відповідність, а 0 — відсутність перетину. Для аудиту конфігурації обчислюють `intersection()` і `union()`, після чого ділять потужність перетину на потужність об'єднання.

1.4.5 Переваги `collections.Counter`

`Counter` є спеціалізованим підкласом `dict` для підрахунку хешованих об'єктів. Він приймає ітерабельні дані, підтримує додавання лічильників та `most_common()`, тому усуває ручну перевірку наявності ключа і спрощує ранжування подій у потоках логів.

1.4.6 Мутувальні та немутувальні операції множин

Методи `update()`, `intersection_update()`, `difference_update()` і `symmetric_difference_update()` змінюють поточну множину на місці. Методи `union()`, `intersection()`, `difference()` та `symmetric_difference()` повертають нову множину. Операції на місці зменшують кількість тимчасових об'єктів, однак їх слід застосовувати лише коли початкові дані більше не потрібні.

1.5 Репозиторій та висновки

Вихідний код лабораторної роботи розміщено у репозиторії: <https://github.com/sobof1/python-labs>, каталог lb6.

У роботі реалізовано інвентаризацію поштових релеїв для варіанта 15. Словник забезпечує зручне моделювання вкладених характеристик, множини відокремлюють дозволені та недозволені адреси, а індекс Жаккара подає ступінь відповідності політиці MX/SPF числовою метрикою. Counter надає компактний спосіб частотного аналізу і визначення найактивніших джерел spam-повідомлень.